

GPU Native Scan — Where We Landed

Branch: `feature/gpu-native-scan-task` (replaces DuckDB's CPU `duckdb_scan_task` with a Sirius-native scan + GPU decode for `.duckdb` files).

Terminology

- **Warm** = 2nd iteration in same session. DuckDB buffer pool populated, OS page cache hot, Sirius GPU pool already allocated. Steady-state query latency.
- **Cold (iter 1)** = first query in a fresh DuckDB process. OS page cache may be warm from prior runs; DuckDB buffer pool empty.
- **Disk-cold** = OS page cache dropped (`echo 3 > /proc/sys/vm/drop_caches`). What post-reboot / container-start looks like.

All warm numbers below are single-session iter-2. Cold numbers specify their tier.

Headline

TPC-H SF=100 on GH200, 22 queries (suite totals):

	GPU	CPU	GPU/CPU
Warm (iter 2)	5.33s	7.08s	1.33× (GPU wins, 15/22 queries)
Cold (disk-cold, lazy mmap — default)	~52.8s	15.5s	0.29× (CPU wins cold)

ClickBench 100-shard on GH200, 29 queries (suite totals):

	GPU	CPU	GPU/CPU
Warm	5.00s	2.54s	0.51×
Cold (disk-cold, lazy mmap)	~95.7s	5.23s	0.05×

Correction to my earlier claim: Cold is *not* roughly equal to CPU at GH200 SF=100. At disk-cold with caches dropped between every query, CPU wins by $\sim 3\times$ on TPC-H and $\sim 18\times$ on ClickBench. CPU scan amortizes page faults inside a vectorized loop over 72 Grace cores; GPU cold still pays the first-pass mmap fault + H2D transfer on top of that. Where cold is roughly equal to CPU: smaller scale on PCIe (TPC-H SF=10 RTX6000 buffer-cold 6.94s vs CPU cold ~ 5 -6s estimated; ClickBench 10M 2.94s vs CPU cold ~ 3 -4s estimated).

Still a huge improvement over where we were: lazy mmap cut GH200 cold from **360s** → **52.8s** (**6.9× win**) vs the old `MADV_POPULATE_READ` prefault that was faulting the whole DB file on first mmap for zero benefit on ATS (commit `27b049e`).

Biggest per-query wins on SF=100 warm:

Q	GPU	CPU	Ratio
Q13	0.332s	0.859s	2.59×
Q22	0.061s	0.148s	2.43×
Q18	0.332s	0.682s	2.05×
Q14	0.101s	0.194s	1.92×

What we shipped

- `gpu_native_scan_task` — bypasses DuckDB's table function for block-backed `.duckdb` storage
- GPU decode kernels: bitpacking (int8/16/32/64), dictionary, FSST, RLE, constant, uncompressed
- Fused GPU-side bitpacking metadata parse (– 26% TPC-H SF10 warm alone)
- Batched two-pass string decode (eliminates per-segment sync + kernel launch)
- Dict/RLE/null-count kernel opts (– 18% ClickBench)
- Direct mmap of `.duckdb` bypassing `BufferManager::Pin()` — **pure Sirius**, atomic fast path, zero mutex contention across parallel scan tasks
- Parallel scan tasks (N concurrent to saturate scan thread pool)

GPU kernel time at SF=100 warm Q1: 608ms — our decode is **11%** of that. Remaining 89% is cuDF `hash_group_by` / CUB internals. We are no longer the bottleneck on compute.

3 open questions

1. Scaling: RTX6000 → GH200, SF=10 → SF=100 — does it compound?

Yes. Direct data from the runs above:

	RTX6000 SF10 warm	GH200 SF100 warm
GPU / CPU	0.53 × (GPU loses)	1.33 × (GPU wins)
15/22 queries →	(GPU wins on 2)	(GPU wins on 15)

The CPU loses its L3-cache advantage as data grows; HBM bandwidth stays flat. GH200's unified-memory ATS also removes the PCIe staging cost entirely, so the scan path gets near its theoretical max. Net: **the cross-over is around SF=100 on GH200 today. Next question is SF=1000.** Does anyone have SF=1000 generated?

2. Bigger-than-memory (PCIe degrades horribly — can't test locally)

On GH200 we're confident: ATS lazy-faults the `.duckdb` mmap, pages stream in at NVLink-C2C speeds. The branch already auto-detects unified memory and skips the prefault step on GH200 (commit `27b049e`), and the 2026-04-16 GH200 run confirmed no warm regression with lazy faults. Cold was **6.9** × **faster** on GH200 with lazy vs prefault (2.4s vs 16.4s avg/query for TPC-H SF100).

What we don't have: an empirical SF=1000 GH200 run with cache flushed. The theory says we stream at line rate. Anyone with cycles to confirm?

On PCIe (RTX6000): unpinned mmap transfers at 6.4 GB/s (already the warm bottleneck at SF=10). Bigger-than-memory is not our problem here — it's the hardware's. We don't plan to optimize for that case.

3. With this speedup, scan/query split: do we need a GPU cache, or push on execution?

Measured split on Q1 SF=10 warm (254ms total):

	ms	%
Scan wall (6 tasks, 2 threads, ~36ms decode/task)	109	43%
Downstream (join / agg / sort)	145	57%

The prior 33/66 estimate was slightly too scan-pessimistic — real split is closer to **43/57** on Q1, and **further toward execution on Q5/Q7/Q9/Q21** (join-heavy). Aggregated across SF=10 warm, **execution dominates**.

Remaining scan optimization (ring buffer + pinned + dual-stream overlap) is worth ~58ms on Q1 SF10 — nice but not huge. **GPU cache of decoded cudf columns** would only win if we see repeated queries against the same table in a session (warm pin is already 1.5ms after DuckDB's buffer pool warms up).

Recommendation: execution first (join / agg / sort downstream), cache later. At SF=100 on GH200, our kernel is 11% of GPU time — cuDF internals are 89%. That's where the next 2× lives. Pushback welcome.

Raw CSVs + nsys profiles: docs/super-sirius/benchmarks/2026-04-16_gh200/ on feature/gpu-scan-duckdb-api. Report: gh200_sf100_clickbench_20260416.md.